

Terminus Agent-Harness Security Review

Date: 2026-08-30 Scope: Terminus effect kernel, native tool-control path, and current security models in Codex, Claude Code, OpenCode, Pi, Hermes Agent, Grok Build, Cline, and Goose Branch:
codex/security-harness-research-20260830 Worktree: /Volumes/Neural/Terminus-security-harness-20260830

Executive decision

Terminus did not make the wrong security architecture. It made the right layers, then failed to connect its autonomy control to an appropriate kernel policy.

Before this review, `full-access` meant only do not ask in the control plane. Native Exec requests still selected the kernel's narrow `secure-local-default` command policy. That policy default-denied ordinary local development tools such as `true`, `sleep`, `make`, project scripts, and many build tools. The result looked like security that was “too good,” but the actual defect was contract mismatch: the UI promised autonomy while the kernel received no authenticated authority to use a broader local command policy.

The correct change is full workspace autonomy, not ambient host autonomy:

- run arbitrary bounded local development commands without interactive approval only when the task contract grants whole-workspace read/write;
- keep execution inside the whole-workspace task authority and enforced OS sandbox;
- keep raw network, ambient secrets, protected Git configuration/hooks, and external-state mutation outside that authority;
- bind the wider command policy to the signed task capability so the caller-supplied intent cannot widen itself;
- reserve any future no-sandbox host mode for a separately reviewed admin contract running behind an outer VM/container boundary.

That change is implemented on this branch. It fixes the immediate inability to work without copying the least safe behavior of competing harnesses.

Scope, method, and evidence standard

The review traced the real native effect path:

permission profile -> task contract -> task capability -> command policy -> approval -> sandbox -> process

It inspected the normative `SPEC.md`, adopted ADRs, kernel/control source, protocol schema, command policy, platform sandboxes, tests, and current UI copy. Competitor claims use primary documentation or their official GitHub repositories, fetched on 2026-08-30. Public documentation cannot prove every closed-source implementation detail; those cases are identified as documented behavior rather than independently verified internals.

Completion evidence for the implementation requires more than compilation: policy unit tests, capability tests, kernel process-start tests, gRPC broker tests, control dispatch tests, UI tests, targeted security/eval suites, broad repository checks, and a final diff review. Repository policy also requires two approvals before any security change merges.

Direct answer: did Terminus already have full access?

Yes in the UI and control plane; no in the effective native command path.

`mini-services/terminus-control/src/permission-profiles.ts` defines `full-access`, `auto`, and `ask`. `full-access` suppresses all control-plane approval prompts. The task contract is still checked first, which is correct.

The failure occurred later:

- 1 `mini-services/terminus-control/src/agent-tools.ts` hard-coded `policyProfileId: "secure-local-default"` for every native process.
- 2 `crates/terminus-kernel/src/services.rs` constructed one `PolicyEngine` from `default_rule_set()` and ignored the profile id for policy selection.
- 3 `policies/command/default.yaml` allowed a small set of test runners and read tools, prompted for `git push`, denied a few known-dangerous cases, and default-denied everything else.
- 4 Kernel prompt approvals used `crates/terminus-kernel/src/approvals.rs`, while UI approval state lived in the control plane. The two were not one durable approval flow. Suppressing the UI prompt therefore could not satisfy a later kernel Prompt decision.

So “Full access” did not mean “the agent can run.” It meant “the control plane will not interrupt before the kernel refuses the command.”

Terminus security posture before the change

What was strong

- The Rust kernel owns process, filesystem mutation, socket, secret, and other effects. Provider/model code cannot directly perform them by design.
- Task capabilities bind principal, session, task, workspace, operation class, resource scope, lifetime, audience, nonce, and optional action hash.
- Command policy is strictest-wins and default-deny.
- macOS uses Seatbelt and Linux uses Bubblewrap when available. The secure profile fails closed if its backend is unavailable.
- Workspace resolution canonicalizes paths and rejects traversal and symlink escape.
- The default sandbox provides a writable workspace while denying general host trees and network, brokering secrets, hiding Terminus state, and write-protecting `.git/hooks`, `.git/config`, and `.git/config.worktree`. On macOS it also retains documented writable package caches and Darwin per-user cache/temp paths; those are a material persistent host surface, not workspace-only containment.
- Bounded output, deadlines, audit events, idempotency, artifacts, and evidence are enforced at the effect boundary.

These controls are materially stronger than approval-only harnesses because a model tricking the approval classifier is not enough to escape the OS boundary.

What was weak or misleading

- 1 Autonomy was not operational. The widest UI setting did not authorize a workable command policy.

- 2 The profile id was unauthenticated authority. `EffectIntent` carried a policy profile string, but the signed token did not bind allowed profiles. Merely teaching the kernel to trust that string would create a confused deputy and let any Exec token request the wider policy.
- 3 Development mode weakened containment. Native agent tools selected `degraded-local` whenever `TERMINUS_DEV=1`. Pairing that with a broad command policy would have turned the fix into host-level arbitrary execution on the most common development path.
- 4 Approvals were split. Kernel Prompt decisions and UI grants were different stores and hashes. A future interactive rule engine needs one durable flow.
- 5 Command effect classification is heuristic. It recognizes direct `curl/wget` reads and `git push`, but arbitrary programs can open sockets. The sandbox, not the classifier, must remain the hard network boundary.
- 6 “Full access” was ambiguous. Other products use the same phrase for a no-sandbox host bypass. Terminus did not, but the UI did not name the difference.
- 7 The macOS sandbox retains persistent host surfaces. Shared tool caches, Darwin cache/temp paths, and broad Mach lookup keep common development tools working, but create cache-poisoning and confused-deputy risk. Allowed package-manager scripts could already reach this surface; a broad command policy makes the tradeoff explicit and demands follow-up hardening.

Competitor comparison

Harness	Unattended mode	Containment boundary	Controls that remain	Security lesson for Terminus
OpenAI Codex	Approval policy never; <code>workspace-write</code> for unattended workspace work; separate <code>danger-full-access / --yolo</code> bypass	macOS Seatbelt, Linux Bubblewrap/seccomp, Windows sandbox; outer container recommended for no-sandbox operation	Approval and sandbox are configured separately; dangerous mode removes both	This is the closest target: default to unattended inside the workspace sandbox, and require explicit outer isolation for host bypass. Official OpenAI security docs
Claude Code	<code>bypassPermissions</code> ; less-permissive <code>acceptEdits</code> , <code>auto</code> , and <code>dontAsk</code> modes	Optional sandboxing uses macOS Seatbelt or Linux/WSL2 isolation	Deny/ask/allow rules and managed settings; documentation recommends container/VM isolation for bypass	Approval bypass must not be described as containment. Managed policy and explicit disable switches are useful operator controls. Permissions , sandboxing
OpenCode	<code>--auto</code> auto-approves ask; configuration can broadly allow	V2 documentation explicitly says Bash is not sandboxed and carries the host user's filesystem/process/network authority	Explicit deny rules still win under <code>--auto</code> ; external-directory and loop checks can ask	Good approval semantics, weak adversarial boundary. Terminus should copy “deny survives auto,” not host execution. Permissions , V2 session spec
Pi	Default tools run without permission popups	Host authority by default; container/sandbox available through an extension or deployment wrapper	Project trust protects loading local resources; extensions can impose policy	Excellent low-friction UX, but containment is delegated to the operator. Terminus can match the UX without delegating the boundary. Official coding-agent README
Hermes Agent	<code>yolo / off</code> , plus smart and manual modes	Local backend is not OS isolation; container/remote backends are the real boundary	Hardline catastrophic rules and deny rules survive yolo; headless dangerous operations fail closed unless explicit	The preserved hard floor is worth copying. Hermes's own security model correctly distinguishes pattern checks from containment. Security guide , security policy
Grok Build	Always-approve / yolo for scripts and CI	Optional OS sandbox, documented as off by default	Deny rules and hooks survive always-approve; managed requirements can disable it	Its approval hierarchy is useful, but optional/fail-open built-in sandbox behavior is below Terminus's fail-closed standard. Permissions , sandbox
Cline	Auto-approve and YOLO can admit files, commands, browser, and MCP without prompts	Isolation is deployment/user responsibility; CLI exposes an optional data-directory sandbox	Per-category auto-approval can be narrower than YOLO	Broad unattended authority is convenient but should be paired with an outer boundary. Auto-approve docs , CLI README
Goose	<code>GOOSE_MODE=auto</code> is documented as the default	Public configuration docs do not establish a mandatory OS sandbox for local commands	Configurable modes; prompt-injection detection exists but is opt-in in the cited configuration	Auto approval and prompt scanning are not substitutes for an effect kernel. Configuration guide

Cross-harness conclusion

Every harness reviewed supports unattended operation. The important split is not “secure versus useful.” It is:

- approval automation: do not stop for a human;
- containment: what the resulting process can reach;
- hard policy: actions that remain denied even when approvals are skipped;
- outer isolation: whether a no-sandbox mode is safe because the entire harness already runs in a disposable VM/container.

Codex and Claude Code document this separation most clearly. Hermes explicitly states that approval/pattern scanning is not containment. Pi, OpenCode, Cline, and local Hermes favor low friction but put more responsibility on the user or deployment boundary. Grok Build preserves deny rules under yolo but documents an optional sandbox with weaker failure behavior than Terminus.

Terminus should not lower its kernel standard to match the least-contained harness. It should make its safe workspace mode actually usable.

Implemented change

1. Contract first

`docs/decisions/ADR-0054-capability-bound-workspace-development-policy.md` records the separation between control-plane approval, command policy, capability authority, and sandbox containment. It explicitly rejects a host bypass in this slice.

2. New command policy

`policies/command/workspace-development.yaml` adds a bounded catch-all for `EXECUTE_LOCAL` and strict denies for classified:

- raw network reads/writes;
- external-state writes;
- secret use and credential administration;
- sandbox/plugin administration;
- download-to-interpreter pipelines.

The policy caps runtime at 600 seconds and inline output at 16 MiB and strips common secret-bearing environment variables. These are defense-in-depth; the secure OS sandbox remains the actual filesystem/network boundary.

3. Signed policy authority

`MintTaskCapabilityRequest` now carries `policy_profile_ids`. The control broker accepts only known, unique ids. `TokenClaims` signs those ids and defaults old tokens to no non-default policy authority.

The kernel resolves `workspace-development` only after validating the Exec token, binder, and scope. A token that lacks the profile receives `PermissionDenied`. An unknown profile receives `InvalidArgument`. Admin capabilities remain the explicit superuser class.

The control broker refuses this profile unless the active task contract grants `**` in both read and write scope. It then signs `**` into the capability rather than pretending the subprocess is confined to its initial `cwd`.

This binding is the security-critical part of the change. Without it, any code holding a narrow Exec token could self-select the wider policy through `EffectIntent`.

4. Native tool wiring

Only native standalone process tools mint the wider profile into their short-lived task capability and request it in the process intent. Reads, patches, verification, scouts, provider commands, and external harness paths retain their current policies.

Native agent Exec, grep, glob, and background jobs now always select the enforced `secure-local-default` sandbox, including under `TERMINUS_DEV=1`. Development mode no longer silently converts unattended native execution into the degraded backend.

Only Exec/background commands select the wider command policy. Read-only grep and glob subprocesses remain on `secure-local-default`. The kernel also rejects any `workspace-development` request paired with `degraded-local` or another sandbox profile, even when the token names the command policy.

5. Truthful UX

The stored id remains `full-access` for protocol compatibility. The displayed label is now Full workspace access with this description:

Runs commands and edits this workspace without asking; host access stays sandboxed.

The label tells the user what becomes autonomous and what does not.

Security properties after the change

Property	Result
Ordinary local commands can run without approval	Yes, for native tools with a profile-bound task token
Task contract can be widened by the profile	No; the profile requires explicit <code>** read/write</code> authority
Caller can self-select the wider policy	No
Workspace scope and symlink/path checks remain	Yes
macOS/Linux enforced sandbox remains	Yes; native tools no longer pick degraded mode in development
Raw command network becomes available	No
Typed <code>web_fetch</code> / brokered network path remains available	Yes, under its separate permission and egress policy
Ambient secrets become available	No
<code>.git/config</code> or hooks become writable	No
General host filesystem becomes writable	No; macOS retains pre-existing allowlisted cache/temp writes, documented below
Existing secure default becomes broader	No
True no-sandbox host mode exists	No; deliberately deferred

Remaining work and recommendations

P0: merge gate for this branch

Do not merge until the targeted security/eval suites and broad repository checks pass and two required reviewers approve the security change. The branch is an implementation candidate, not self-authorizing production policy.

P1: unify the approval system

Replace the control-plane waiter/grant state and kernel `ApprovalStore` with one durable, exact-hash approval lifecycle. A kernel `Prompt` should emit one approval record the UI can decide, and the kernel should consume that exact decision. Restart, replay, expiry, scope, and task ownership need integration tests. Until then, broader profiles should prefer hard allow or hard deny over kernel prompts that the selected UI mode cannot satisfy.

P1: eliminate implicit degraded execution elsewhere

The native agent path is fixed, but other development-only provider, verification, and control routines still name `degraded-local`. Audit each one and require either:

- the enforced sandbox;
- a test-only process boundary that cannot receive user workloads or secrets; or
- an explicit operator flag whose UI/audit record says isolation is degraded.

`TERMINUS_DEV=1` alone should not be authority to weaken a production-like effect path.

P1: add a packaged end-to-end autonomy cohort

Run the actual control + kernel + desktop/native task path and prove:

- 1 Full workspace access runs `/usr/bin/true`, `sleep`, a project-local binary, a build, and a test without approval.
- 2 Ask mode presents one control approval and then runs the same command.
- 3 An unbound token cannot select `workspace-development`.
- 4 A process cannot read `~/ .ssh`, write outside the workspace, modify `.git/hooks` or `.git/config`, open a raw socket, or receive an ambient provider key.
- 5 A missing Seatbelt/Bubblewrap backend fails closed and the UI explains why.
- 6 Background jobs enforce the same token, policy, and sandbox after restart.

P1: harden the macOS persistent host surface

The existing Seatbelt profile admits writes to shared package caches and Darwin per-user cache/temp paths, and permits broad Mach lookup. This is still an OS sandbox, but it is not an immutable-host boundary. Move writable caches to per-workspace or disposable storage, narrow Mach services to a tested allowlist, and add live probes for cache poisoning, Keychain/clipboard access, Apple Events, launch services, and persistence. Until then, treat hostile repositories as capable of modifying allowlisted caches and use an outer disposable VM/container for high-risk workloads.

P2: design host bypass only as an admin isolation contract

If Terminus eventually needs Codex-style `danger-full-access`, do not implement it as a fourth permission-profile string. Make it a separate admin contract requiring:

- an explicit one-session choice with strong warning copy;
- proof that the harness is already inside an approved disposable VM/container or an equivalent host boundary;
- no ambient secret inheritance by default;
- durable audit and revocation;
- managed-policy ability to disable it;
- adversarial escape, persistence, and credential-exposure evaluation;
- security-owner approvals before merge and enablement.

Without outer isolation, “no prompts and no sandbox” is simply arbitrary code execution as the signed-in user. That may be an intentional operator choice, but it is not a safe default and it is not necessary to make Terminus useful.

P2: make profile resolution a first-class registry

This vertical slice intentionally adds one non-default profile. If more are added, replace the two-engine selection with a typed registry that owns profile ids, ruleset hashes, compatibility versions, capability eligibility, and audit fields. Keep unknown ids fail-closed and preserve the secure default for legacy tokens.

Final assessment

The kernel was not “too secure.” Terminus had a product-level autonomy control that ended before the kernel, plus a kernel that had only a reviewer-grade command allowlist. The safe fix was to connect them with authenticated, least-authority policy selection.

With this branch, Full workspace access can become genuinely hands-off for normal local coding while Terminus keeps the part competitors most often leave to deployment discipline: a non-bypassable effect boundary. The remaining high-priority work is approval unification, removal of other implicit degraded paths, packaged end-to-end proof, and two-reviewer security approval. A raw host bypass is optional future admin functionality, not a prerequisite for agent autonomy.

Source ledger

Terminus sources

- `SPEC.md §13, §26, §31, §36`
- `docs/decisions/ADR-0045-declarative-tool-permission-engine.md`
- `docs/architecture/effect-kernel.md`
- `mini-services/terminus-control/src/permission-profiles.ts`
- `mini-services/terminus-control/src/index.ts`
- `mini-services/terminus-control/src/agent-tools.ts`
- `crates/terminus-authz/src/token.rs`
- `crates/terminus-kernel/src/services.rs`
- `crates/terminus-kernel/src/approvals.rs`
- `crates/terminus-policy/src/engine.rs`
- `policies/command/default.yaml`
- `crates/terminus-sandbox/src/profile.rs`
- `crates/terminus-sandbox-macos/src/lib.rs`
- `crates/terminus-sandbox-linux/src/lib.rs`

External primary sources

- [OpenAI Codex approvals and security](#)
- [OpenAI Codex permissions](#)
- [Claude Code permissions](#)
- [Claude Code sandboxing](#)
- [OpenCode permissions](#)
- [OpenCode V2 session security note](#)
- [Pi coding agent README](#)
- [Hermes security guide](#)
- [Hermes security policy](#)

- [Grok Build permissions and safety](#)
- [Grok Build sandbox](#)
- [Cline auto-approve](#)
- [Cline CLI README](#)
- [Goose configuration guide](#)

Limitations

- This review reflects sources available on 2026-08-30. Harness defaults and flags change quickly.
- Public documentation describes intended behavior; it is not proof against implementation defects or sandbox escapes.
- No live credentialed provider calls or production environments were touched.
- A full packaged desktop end-to-end run is a separate acceptance gate and is not replaced by unit/integration tests.